server checks that the form data contains this hidden field and that the value matches the value stored in the session associated with the cookie. Any form data that is received without the hidden field is rejected. This effectively prevents CSRF attacks because an attacker cannot guess the random fields and so cannot forge a correct request.

An API does not have the luxury of adding hidden form fields to requests because most API clients want JSON or another data format rather than HTML. Your API must therefore use some other mechanism to ensure that only valid requests are processed. One alternative is to require that calls to your API include a random token in a custom header, such as `X-CSRF-Token`, along with the session cookie. A common approach is to store this extra random token as a second cookie in the browser and require that it is sent as both a cookie and an `X-CSRF-Token` header on each request. This second cookie is not marked HttpOnly, so that it can be read from JavaScript (but only from the same origin). This approach is known as a *double-submit cookie*, as the cookie is submitted to the server twice. The server then checks that the two values are equal as shown in figure 4.11.

> **DEFINITION** A ***double-submit cookie*** is a cookie that must also be sent as a custom header on every request. As cross-origin scripts are not able to read the value of the cookie they cannot create the custom header value, so this is an effective defense against CSRF attacks.

This traditional solution has some problems, because while it is not possible to read the value of the second cookie from